

Compiler Implementation Project – Checkpoint 3 Report

Nathan Brommersma, Ahmad Sawan, Jacob McKenna

April 2, 2025

CIS*4650 – Dr. Fei Song

1. Introduction

This report outlines information on checkpoint 3 of our compiler implementation project.

Checkpoint 3 added a Code Generator to our source code, which can generate working assembly code for the TMSimulator. This report covers the topics of: Implementation, Design Process and Testing, Lessons learned and Limitations, Significant changes to previous checkpoints, Possible improvements, and Team Member Contributions. At the end of this report the reader should have a strong understanding of the implementation details for our Checkpoint 3.

2. Implementation Overview

For our checkpoint 3 implementation we added the Code Generator (`CodeGenerator.java`) class to our source code, which is responsible for converting our abstract syntax tree to runnable assembly code. The Code Generator was implemented with many variables to keep track of registers and memory offsets. Having these variables at the class level is important to ensure that offsets are consistent, and memory locations are not lost for variables and functions. There are several helper classes that were available to use from the course slides to help us create our assembly code, providing functions to quickly write instructions. We used these helper functions in all our visitor methods as well as for generating the prelude and finale of our assembly code. Alongside the Code Generator, we

had to make slight changes to some of the existing abstract syntax tree classes (5. Significant Changes to Previous Checkpoints). This change would be important to keep track of variable offsets and function addresses during code generation. At the end of this checkpoint, we were able to get a large amount of the implementation completed successfully.

3. Design Process and Testing

3.1 Design Process

For checkpoint 3 we followed a similar design and planning process to previous checkpoints. We followed the checkpoint 3 grading scheme to create our list of requirements that we needed to complete. We designed our Code Generator to extend off our existing abstract syntax tree visitor classes, as was recommended in class. In our Code Generator class, we used a variety of helper functions to help us create the instructions that we would use for each visitor class, such as “emitRM” and “emitRM_Abs”. We then override each visitor method to generate the correct instructions for each C-minus part of code. The Code Generator uses the visitor design pattern, which we have found to be extremely useful for code generation, as it allows us to use our existing tree with only minor changes.

3.2 Testing

All testing was performed on the linux.socs.uoguelph.ca Linux server for a controlled environment.

Control Structure Testing: For testing control structures, we primarily used the provided `fac.cm` and `booltest.cm` files. These two files tested the basic functionality of “if” and the “while” loop. All control structure testing was successful.

Functions and Recursions: To test functions and recursions, we relied on the two files given from earlier checkpoints: “gcd.cm” and “mutual.cm”. These two programs tested functions and self-recursion, as well as function prototypes. We were able to get the full “gcd.cm” code working; however, we came across an issue with function prototypes while testing “mutual.cm” (More info in 4.2 Limitations).

Prelude and Finale, Arithmetic, Assignments, Nested Blocks in Functions: Our strategy for testing the other parts of the program were through individual testing files that we created for each part. For example, to test the prelude and finale, we created a program with an empty main and examined the output to ensure that the prelude and finale code generation was working as expected. We followed similar tests for the other portions of the implementation.

4. Lessons Learned and Limitations

4.1 Lessons Learned

Functions: During our Code Generation implementation, we struggled a lot with getting our functions to return to the caller properly. This issue was because we were not saving the return address as the first thing we did when jumping to the function, leaving us to be unable to return when we reached the end of the function. We read over the slides once again and realized that we had to save the return address from register 0 as the first thing

after we jump and then save it to memory. After we implemented this, we were able to run our functions and recursive function testing programs successfully.

Second Symbol Table: In our Code Generator Class (CodeGenerator.java), we added an additional symbol table to keep track of variable information and function data. In hindsight to our implementation, it might have been more efficient to use the pre-existing symbol table that we generated for checkpoint 2. We did not do that right away because we wanted to get the basic functionality of code generation working before optimizing the code. In the future this is one section of our checkpoint that we should have spent more time designing.

4.2 Limitations

Array Implementation: Our code generation implementation was not able to get array functionality working. This includes all programs that use any type of integer array, the resulting generated code will not function as expected. We tried to get arrays working but could not figure out all the issues before submission. Another part that will not work because of this is index out of bounds errors.

Function Prototypes: The code generation of our compiler does not have a full implementation of function prototypes, which is a blocker to some functionality with function calling. Functions must only be called after they are defined, as the function prototypes were not implemented correctly. If a function prototype is in the code, our compiler will report it as an error.

5. Significant Changes to Previous Checkpoints

Augmentation to checkpoint 2 absyn classes: As per the checkpoint 3 tips, we added some helpful variables to certain absyn files that would help with the code generation.

Some useful examples include: “funaddr” to function declarations, and “offset” to variables. We were able to utilize these changes in the code generation to keep track of the function address in the assembly, as well as the offset that we assign to variables.

6. Possible Improvements

Code Optimization: The main way to improve this compiler’s implementation is through optimizing the code generation to decrease the size of the output C-minus file. There are a few ways to accomplish this, but the easiest would be to increase the number of used registers and remove tail recursion from recursive programs. Utilizing more registers would decrease the number of things we have to store in memory, thus reducing the number of instructions generated. Similarly, removing tail recursion would greatly reduce the amount of function call overhead, which need less operations.

7. Team Member Contributions

All team members contributed to this checkpoint to complete the implementation of the code generator.

Ahmad Sawan: Worked on the Code Generator with peer programming and contributed to the documentation.

Nathan Brommersma: Worked on the Code Generator with peer programming and contributed to the documentation.

Jacob McKenna: Worked on the Code Generator with peer programming and contributed to the documentation.

8. Conclusion

In checkpoint 3 we have completed the final part of our compiler implementation and have finished a working compiler for the C-Minus language. We have used what we have learned from the last months lectures to create the Code Generator, which works alongside our parser and semantic analyzer to produce runnable code for the TMSimulator. While some areas of the compiler need to be improved such as the array handling and function prototypes, we are proud of what we were able to accomplish and learn this semester. Being able to put the main pieces of a compiler together has been rewarding and a great learning experience.